Natural Language Processing **Tel Aviv University**

Assignment 3: Transformers & IR

Due Date: *January 18th, 2026* Lecturer: Dr. Tal Wagner, TA: Idan Tarshish

Instructions

In this exercise, questions 1-4 are mandatory, and question 5 is a bonus question.

For these questions, as well as for your final project, you will likely need to use HuggingFace's packages. If you aren't familiar with them yet, you can use the following online resources: brief introductory tutorial; deeper dive course.

1 Attention Exploration (30 pt)

Multi-head self-attention is the core modeling component of Transformers. In this question, we'll get some practice working with the self-attention equations, and motivate why multi-headed self-attention can be preferable to single-headed self-attention.

Recall that attention can be viewed as an operation on a *query* vector $q \in \mathbb{R}^d$, a set of *value* vectors $\{v_1, \ldots, v_n\}, v_i \in \mathbb{R}^d$, and a set of *key* vectors $\{k_1, \ldots, k_n\}, k_i \in \mathbb{R}^d$, specified as follows:

$$c = \sum_{i=1}^{n} v_i \alpha_i \tag{1}$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)} \tag{2}$$

with $\alpha = \{\alpha_1, \ldots, \alpha_n\}$ termed the "attention weights". Observe that the output $c \in \mathbb{R}^d$ is an average over the value vectors weighted with respect to α.

1. **Copying in attention.** One advantage of attention is that it's particularly easy to "copy" a value vector to the output c. In this problem, we'll motivate why this is the case.
   (a) **Explain** why α can be interpreted as a categorical probability distribution.
   (b) The distribution α is typically relatively "diffuse"; the probability mass is spread out between many different α_i. However, this is not always the case. **Describe** (in one sentence) under what conditions the categorical distribution α puts almost all of its weight on some α_j, where $j \in \{1, \ldots, n\}$ (i.e. $\alpha_j \gg \sum_{i \neq j} \alpha_i$). What must be true about the query q and/or the keys $\{k_1, \ldots, k_n\}$?
   (c) Under the conditions you gave in (b), **describe** the output c.
   (d) **Explain** (in two sentences or fewer) what your answers to (b) and (c) mean together, intuitively.
2. **An average of two.** Instead of focusing on just one vector v_j, a Transformer model might want to incorporate information from *multiple* source vectors. Consider the case where we instead want to incorporate information from **two** vectors v_a and v_b, with corresponding key vectors k_a and k_b.

(a) How should we combine two d-dimensional vectors v_a, v_b into one output vector c in a way that preserves information from both vectors? In machine learning, one common way to do so is to take the average: $c = \frac{1}{2}(v_a + v_b)$. It might seem hard to extract information about the original vectors v_a and v_b from the resulting c, but under certain conditions one can do so. In this problem, we'll see why this is the case.

Suppose that although we don't know v_a or v_b, we do know that v_a lies in a subspace A formed by the m basis vectors $\{a_1, a_2, \ldots, a_m\}$, while v_b lies in a subspace B formed by the p basis vectors $\{b_1, b_2, \ldots, b_p\}$. (This means that any v_a can be expressed as a linear combination of its basis vectors, as can v_b. All basis vectors have norm 1 and are orthogonal to each other.) Additionally, suppose that the two subspaces are orthogonal; i.e. $a_j^\top b_k = 0$ for all j, k.

Using the basis vectors $\{a_1, a_2, \ldots, a_m\}$, construct a matrix M such that for arbitrary vectors $v_a \in A$ and $v_b \in B$, we can use M to extract v_a from the sum vector $s = v_a + v_b$. In other words, we want to construct M such that for any v_a, v_b, $Ms = v_a$. Show that $Ms = v_a$ holds for your M.

Hint: First, come up with a rank-1 matrix A_i such that $A_i a_i = a_i$ and $A_i b = 0$ for every b such that $b^\top a_i = 0$. Now, use those A_is to construct the desired matrix M.

(b) As before, let v_a and v_b be two value vectors corresponding to key vectors k_a and k_b, respectively. Assume that (1) all key vectors are orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1.[1] **Find an expression** for a query vector q such that $c \approx \frac{1}{2}(v_a + v_b)$, and justify your answer. [2]

3. **Drawbacks of single-headed attention:** In the previous part, we saw how it was *possible* for a single-headed attention to focus equally on two values. The same concept could easily be extended to any subset of values. In this question we'll see why it's not a *practical* solution. Consider a set of key vectors $\{k_1, \ldots, k_n\}$ that are now randomly sampled, $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, where the means $\mu_i \in \mathbb{R}^d$ are known to you, but the covariances Σ_i are unknown. Further, assume that the means μ_i are all perpendicular; $\mu_i^\top \mu_j = 0$ if $i \neq j$, and unit norm, $\|\mu_i\| = 1$.

(a) Assume that the covariance matrices are $\Sigma_i = \alpha I, \forall i \in \{1, 2, \ldots, n\}$, for vanishingly small α. Design a query q in terms of the μ_i such that as before, $c \approx \frac{1}{2}(v_a + v_b)$, and provide a brief argument as to why it works.

(b) Though single-headed attention is resistant to small perturbations in the keys, some types of larger perturbations may pose a bigger issue. Specifically, in some cases, one key vector k_a may be larger or smaller in norm than the others, while still pointing in the same direction as μ_a. As an example, let us consider a covariance for item a as $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ for vanishingly small α (as shown in figure 1). This causes k_a to point in roughly the same direction as μ_a, but with large variances in magnitude. Further, let $\Sigma_i = \alpha I$ for all $i \neq a$.

When you sample $\{k_1, \ldots, k_n\}$ multiple times, and use the q vector that you defined in part (a), what do you expect the vector c will look like qualitatively for different samples? Think about how it differs from part (a) and how c's variance would be affected.

4. **Benefits of multi-headed attention:** Now we'll see some of the power of multi-headed attention. We'll consider a simple version of multi-headed attention which is identical to single-headed self-attention as we've presented it in this homework, except two query vectors (q_1 and q_2) are defined,

[1] Recall that a vector x has norm 1 iff $x^\top x = 1$.

[2] Hint: while the softmax function will never *exactly* average the two vectors, you can get close by using a large scalar multiple in the expression.

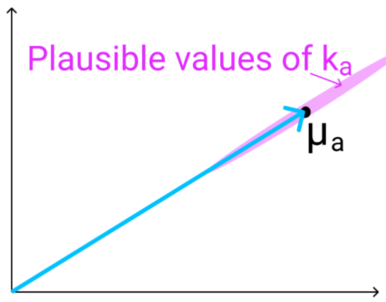

Figure 1: The vector μ_a (shown here in 2D as an example), with the range of possible values of k_a shown in red. As mentioned previously, k_a points in roughly the same direction as μ_a, but may have larger or smaller magnitude.

which leads to a pair of vectors (c_1 and c_2), each the output of single-headed attention given its respective query vector. The final output of the multi-headed attention is their average, $\frac{1}{2}(c_1 + c_2)$. As in question 1(3), consider a set of key vectors $\{k_1, \ldots, k_n\}$ that are randomly sampled, $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, where the means μ_i are known to you, but the covariances Σ_i are unknown. Also as before, assume that the means μ_i are mutually orthogonal; $\mu_i^\top \mu_j = 0$ if $i \neq j$, and unit norm, $\|\mu_i\| = 1$.

(a) Assume that the covariance matrices are $\Sigma_i = \alpha I$, for vanishingly small α. Design q_1 and q_2 such that c is approximately equal to $\frac{1}{2}(v_a + v_b)$. Note that q_1 and q_2 should have different expressions.

(b) Assume that the covariance matrices are $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ for vanishingly small α, and $\Sigma_i = \alpha I$ for all $i \neq a$. Take the query vectors q_1 and q_2 that you designed in part i. What, qualitatively, do you expect the output c to look like across different samples of the key vectors? Explain briefly in terms of variance in c_1 and c_2. You can ignore cases in which $k_a^\top q_i < 0$.

Acknowledgements This question was adapted from Stanford's CS224n course. Their contributions are greatly appreciated.

2 In-Context Learning (35 pt)

Through this question, you will experience using `GPT-2`, a transformer-based decoder only model, and explore its performance within a few and zero-shot learning settings. You will further compare its performance to that of `GPT-3`.

To evaluate that, we will use `TriviaQA` - a popular Question Answering benchmark, which is very commonly used to evaluate current LMs question answering abilities. This benchmark contains a few thousands of trivia questions, such as *"In which country is Lake Como?"* or *"In the British monarchy, who succeeded Queen Anne to the throne?"*, formulated as a multiple-choice question. We're though not going to stick to this format, but apply our evaluation within an open domain closed-book questions. That is, we will rely on the model's current factual knowledge, to be extracted while attempting to answer a given question.

Please upload the `in_context.ipynb` file to google colab, and answer the following questions by filling in your code.

Important! Please enable a GPU in your Colab environment setting, and make sure you run your models on this.

1. **Random Demonstrations**. We're first going to explore the performance given a random set of increasing sizes of in-context demonstrations.

   (a) `optional_in_context_demonstrations` is a list of 500 examples from `TriviaQA`. Each example is represented as a tuple, where the first entry is a question, as the second is a list of possible answers. Here you're required to randomly sample small sets of in-context examples, of sizes 3-8. Formally, you should have 6 different sets - the first one is of size 3, the second of size 4, ..., the last of size 8. Use each of these sets to construct an in-context prompt. This prompt should be phrases as follows:

   Question: ¡first question¿
   Answer: ¡one of its possible answers¿
   ...
   Question: ¡last question¿
   Answer: ¡one of its possible answers¿

   Now, evaluate the performance of `GPT2-medium`, using each one of these prompts, on the validation set (`validation_set`), by using the function `check_answer_truthfulness`. In this section, please apply beam search decoding (`beam_search` function).

   Please report the results (correctness average over the whole validation set), using `pandas.Dataframe`. You just have to print it :)

   (b) Now apply the same process, but using sampling decoding (with temperature = 0.7). Try to explain the differences in results in two sentences.

2. **Demonstrations Retrieval**. In this section, we will retrieve the examples that are most "semantically" similar to the test question, out of `optional_in_context_demonstrations`, according to `BERT`.

   (a) Use `encode_question` to first encode the questions of all the examples in `optional_in_context_demonstrations`. Then, for each of the examples in the validation set, find the closest 8 examples in `optional_in_context_demonstrations` - here our measure to similarity is the dot product between the encoding vector of the test question, to this of the "train" question. Note that now, each of our validation examples will be accompanied with a different prompt. All that left is to evaluate and report the new results - both with beam search, and sampling decoding.

   (b) Explore a batch of examples from the validation set, and their new prompts. Can you explain why results have got better compared to the those of the random prompts from previous sections?

3. **Zero-shot**. Now we will try a zero-shot setting, where we don't give the model any demonstrations, but directly asking it the question.

   (a) Evaluate and report the results on the given validation set.

   (b) Have a look on the model's generations from the last section. Try to explain why the results are that low.

(c) In order to solve this, we will use `LAMA`, which is a known sentence-completion benchmark, contains of many facts taken from Wikipedia. In This benchmark, the query is phrased as a fill-in-the-blank sentence, where the missing text is the target answer. For instance, *"It is found in north-western Russia, Ukraine, Romania, Bulgaria, Greece and"*, while the answer is *"Italy"*. Evaluate and report the results on `lama_validation_set`, which is a list of tuples composed the same way as the validation set of `TriviaQA`.

(d) Do you think the results are actually higher than those you reported? Take a look on the model's generations and try to explain.

3 Textual Search (code implementation) (30 pt)

We have discussed textual search as an NLP task in various points throughout the class. The goal of textual search is, given a textual query, to search the most relevant text passages in a large corpus. An important example comes from Question Answering - many modern QA systems are a compound of an information retrieval (IR) model and a reading comprehension (RC) model. The purpose of the IR model is, given the question that has been asked, to find the most relevant passages appearing in the web (e.g. Wikipedia), which are likely to contain the answer. Then, the purpose of the RC model is to extract the specific answer out of them.

In this exercise you will explore two widely used NLP methods for textual search: sparse search and dense search. Please upload the `textual_search.ipynb` file into colab. First, run the cells of code which download and construct the corpus. Then, implement the code according to the following instructions.

3.1 Sparse search with TF-IDF

Sparse search is largely based on keyword matching between the query and the documents in the corpus. To quantify the relevance of a document in the corpus to the query, a common measure is TF-IDF, which stands for Term Frequency-Inverse Document Frequency.

In this exercise, you may need to divide sentences into words. You are not allowed to use existing functions in any package for this (such as tokenizers) and must implement a function yourself. However, you may use packages like re and numpy.

We will use t to refer a single token/word. C will stand as our corpus, which is formally a set of documents. We will use d to denote a single document from the corpus.

1. Term Frequency (TF). Suppose we have a set of English text documents and wish to rank which document is most relevant to the query , "What is the capital of New Zealand?". A simple way to start out is by eliminating documents that do not contain all three words "capital","New", and "Zealand", but this might still leave many irrelevant documents (depending on our corpus). To further distinguish between them, we might count the number of times each term occurs in each document; the number of times a term occurs in a document is called its term frequency, and denoted by TF. Formally, if we denote the number of times t appears in d as $count(t, d)$, and the total number of tokens in d by $size(d)$, we get: $TF(t, d) = \frac{count(t,d)}{size(d)}$.

   Implement the function `computeTF`, which given a token and a document, computes the corresponding TF measure.

2. Document Frequency (DF). This measures the importance of document in whole set of corpus, this is very similar to TF. The only difference is that TF is frequency counter for a term t in document d, where as DF is the count of occurrences of term t in the document set C. In other words, DF is the number of documents in which the word is present. Formally, $DF(t, N) = \sum_{d \in N} 1_{t \in d}$.[3] We add one occurrence if the term appears in a document at least once, we do not need to know the number of times the term appears.

   Implement the function `computeDF` which given a token, and a corpus (which is a list of documents), computes the token's DF.

3. Inverse Document Frequency (IDF). While computing TF, all terms are considered equally important. However it is known that certain terms, such as "is", "of", and "that", may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones which may be more indicative of related documents when they co-occur, by computing the IDF, an inverse document frequency factor is incorporated which diminishes the weight of terms that occur very frequently in the document set and increases the weight of terms that occur rarely. IDF is the inverse of the document frequency which measures the informativeness of term t. When we calculate IDF, it will be very low for the most frequent words such as stop words (because stop words such as "is" is present in almost all of the documents and thus give very little information when they appear, and $\frac{size(C)}{DF}$ will assign a very low value for that word). This finally gives what we want, a relative weighting. Formally, we would like to define it as: $IDF(t) = \frac{size(C)}{DF(t)}$ Now there are few other problems with this, in case of a large corpus, say one with 100,000,000 documents, the IDF value explodes. To avoid the effect we take the log of IDF. During the query time, when a word which is not in vocabulary appears, the DF will be 0. As we cannot divide by 0, we smooth the value by adding 1 to the denominator. Thus, we arrive to the final formula: $IDF(t) = \log \frac{size(C)}{DF(t)+1}$.

   Implement the function `computeIDF` which given a token and a corpus, computes the corresponding IDF of the token.

4. We are finally ready to compute $TF - IDF$, our measure that evaluates how important a word is to a document in a corpus. There are many different variations of $TF - IDF$ but for now let us concentrate on the this basic version. $TFIDF(t, d) = TF(t, d) * IDF(t, C)$.

   Implement the function `computeTFIDF` which given a token, a document and a corpus, computes the corresponding $TFIDF$ of the token and the document.

5. **Query mechanism:** Write a function that receives a textual query and a textual corpus and returns the top-3 documents with the highest TF-IDF scores, using the function you've implemenet above.

3.2 Dense search with encoder models

Dense search takes a different approach to textual search. It embeds all of the documents in the corpus into vectors in $\mathbb{R}^d$ (thus creating a dataset of vectors), then embeds the query in $\mathbb{R}^d$ in the same way, and then measures the relevance of a document to the query by a geometric similarity measure between their vector embeddings. Some commonly used geometric similarity measures are the Euclidean distance, the cosine similarity, and the inner product. In this exercise we will use the inner product.

[3] 1_b is 1 when the boolean condition b is true, and 0 otherwise.

We will embed the texts in $\mathbb{R}^d$ using a pre-trained transformer-based encoder-only model similar to BERT which we have seen in class. The specific model we will use is `all-mpnet-base-v2` which is available in the `sentence_transformers` library. The model is built on the MPNet architecture and is finetuned using the Sentence-BERT method. The embedding dimension is $d = 768$.

1. **Ingestion:** First, we will "ingest" the corpus into a vector dataset. This part runs only once prior to receiving any queries. Then the pre-ingested dataset can be used for all queries.

   Write the code that loads the `all-mpnet-base-v2` embedding model and embeds the textual corpus we have created above into a numpy matrix called `embedded_corpus`. The matrix should have a row per document in the corpus and $d = 768$ columns. Each row should be set to the vector embedding of the corresponding document in the corpus.

2. **Query mechanism:** Write a function that received a textual query and the embedded corpus, embeds the query into a vector with the same embedding model, and returns the top-3 documents in the corpus as measured by the inner product between their vectors and the query vector.

   One advantage of dense search with vectors is the parallelism possible with vector operations. Make sure that your query mechanism is vectorized. Iterative solutions (that compute the inner products one by one) will receive only partial credit.

3.3 What you need to submit

Include the code you've written in your submission. In addition, in the included PDF, include the answers to the following questions:

1. Consider the following three queries:

   (a) *What is the capital of New Zealand?*

   (b) *What currency is used in New Zealand?*

   (c) *Should I visit New Zealand if I am interested in sightseeing?*

   For each query, list the top-3 results with sparse search and the top-3 results with dense search, along with their scores (TF-IDF scores for sparse search, and inner product scores for dense search).

2. Sparse and dense search both have pros and cons. In what situations would you expect sparse search to be better and in what situations would you expect dense search to be better?

   - In this question, refer only to the quality and relevance of the search results, and disregard computational aspects (i.e., which method is more efficient).
   - **Answer is no more than 4 sentences.**

4 AI Disclosure & Reflection (5 pt)

The following question is graded only for its completion, not its content, nor for whether you used AI or not. Furthermore, its content does not impact the grading and evaluation of the rest of your submission. The goal is to promote transparency and reflection on LLM and AI usage.

Answer the following parts briefly (1-2 paragraphs):

1. Did you use AI in the work on this assignment? If yes, please name the specific tools/models used (e.g., ChatGPT—model name, Copilot, Claude, etc.).
2. How and why did you use it – which parts of the work and for what purpose (e.g., brainstorming, outlining, code generation, debugging, editing, evaluation).
3. Reflection: What helped or hindered? What is your takeaway from this experience?

If you did not use AI, write: "No AI used", and add a paragraph explaining why.

Please refer in your reflection also to the bonus question (Q5) if you chose to do it

5 Pretrained Transformers (Bonus Question)

In this section you'll be training a Transformer to perform a task that involves accessing knowledge about the world — knowledge which isn't provided via the task's training data (at least if you want to generalize outside the training set). You'll find that it more or less fails entirely at the task. You'll then learn how to pretrain that Transformer on Wikipedia text that contains world knowledge, and find that finetuning that Transformer on the same knowledge-intensive task, might be helpful for enabling the model to access some of the knowledge learned at pretraining time. You'll find that this enables models to perform considerably OK on a held out validation set.

Through this section, you will fill in your code within the `pretrained_transformers` directory, and write your textual answers within the pdf file. You will probably want to develop on your machine locally, then run training on Slurm/Colab. You'll also probably need a few hours for the training, so plan your time accordingly!

1. **Check out the demo.**
   In the `mingpt-demo/` folder is a Jupyter notebook `play_char.ipynb` that trains and samples from a Transformer language model. Take a look at it (either locally on your computer or on Colab) to get somewhat familiar with how it defines and trains models. Some of the code you're writing below will be inspired by what you see in this notebook.

   Note that you do not have to write any code or submit written answers for this part.

2. **Read through `NameDataset` in `src/dataset.py`, our dataset for reading name-birthplace pairs.**
   The task we'll be working on with our pretrained models is attempting to access the birth place of a notable person, as written in their Wikipedia page. We'll think of this as a particularly simple form of question answering:

   Q: Where was [person] born?
   A: [place]

   From now on, you'll be working with the `src/` folder. **The code in `mingpt-demo/` won't be changed or evaluated for this assignment.** In `dataset.py`, you'll find the the class `NameDataset`, which reads a TSV (tab-separated values) file of name/place pairs and produces examples of the above form that we can feed to our Transformer model.

   To get a sense of the examples we'll be working with, if you run the following code, it'll load your `NameDataset` on the training set `birth_places_train.tsv` and print out a few examples.

```
python src/dataset.py namedata
```

Note that you do not have to write any code or submit written answers for this part.

3. **Implement finetuning (without pretraining).**

   Take a look at `run.py`. It has some skeleton code specifying flags you'll eventually need to handle as command line arguments. In particular, you might want to *pretrain*, *finetune*, or *evaluate* a model with this code. For now, we'll focus on the finetuning function, in the case without pretraining.

   Taking inspiration from the training code in the `play_char.ipynb` file, write code to finetune a Transformer model on the name/birthplace dataset, via examples from the `NameDataset` class. For now, implement the case without pretraining (i.e. create a model from scratch and train it on the birthplace prediction task from part (b)). You'll have to modify two sections, marked `[part c]` in the code: one to initialize the model, and one to finetune it. Note that you only need to initialize the model in the case labeled "vanilla" for now (later in section (g), we will explore a model variant). Use the hyperparameters for the `Trainer` specified in the `run.py` code.

   Also take a look at the *evaluation* code which has been implemented for you. It samples predictions from the trained model and calls `evaluate_places()` to get the total percentage of correct place predictions. You will run this code in part (d) to evaluate your trained models.

   This is an intermediate step for later portions, including Part 4, which contains commands you can run to check your implementation. No written answer is required for this part.

4. **Make predictions (without pretraining).**

   Train your model on `birth_places_train.tsv`, and evaluate on `birth_dev.tsv`. Specifically, you should now be able to run the following three commands:

```
# Train on the names dataset
python src/run.py finetune vanilla wiki.txt \
        --writing_params_path vanilla.model.params \
        --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set, writing out predictions
python src/run.py evaluate vanilla wiki.txt  \
        --reading_params_path vanilla.model.params \
        --eval_corpus_path birth_dev.tsv \
        --outputs_path vanilla.nopretrain.dev.predictions

# Evaluate on the test set, writing out predictions
python src/run.py evaluate vanilla wiki.txt  \
        --reading_params_path vanilla.model.params \
        --eval_corpus_path birth_test_inputs.tsv \
        --outputs_path vanilla.nopretrain.test.predictions
```

   Training should take less than 30 minutes. Report your model's accuracy on the dev set (as printed by the second command above). We also have Tensorboard logging for debugging. It can be launched using `tensorboard --logdir expt/`. Don't be surprised if it is well below 10%; we will be digging into why in the following sections. As a reference point, we want to also calculate the accuracy the model would have achieved if it had just predicted "London" as the birth place for everyone in the dev set. Fill in `london_baseline.py` to calculate the accuracy of that approach and report your result in your write-up. You should be able to leverage existing code such that the file is only a few lines long.

5. **Define a *span corruption* function for pretraining.**
In the file `src/dataset.py`, implement the `__getitem__()` function for the dataset class `CharCorruptionDataset`. Follow the instructions provided in the comments in `dataset.py`. Span corruption is explored in the T5 paper. It randomly selects spans of text in a document and replaces them with unique tokens (noising). Models take this noised text, and are required to output a pattern of each unique sentinel followed by the tokens that were replaced by that sentinel in the input. In this question, you'll implement a simplification that only masks out a single sequence of characters.

   To help you debug, if you run the following code, it'll sample a few examples from your `CharCorruptionDataset` on the pretraining dataset `wiki.txt` and print them out for you.

```
python src/dataset.py charcorruption
```

   No written answer is required for this part.

6. **Pretrain, finetune, and make predictions. Budget 2 hours for training.**
Now fill in the *pretrain* portion of `run.py`, which will pretrain a model on the span corruption task. Additionally, modify your *finetune* portion to handle finetuning in the case *with* pretraining. In particular, if a path to a pretrained model is provided in the bash command, load this model before finetuning it on the birthplace prediction task. Pretrain your model on `wiki.txt` (which should take approximately two hours), finetune it on `NameDataset` and evaluate it. Specifically, you should be able to run the following four commands: (Don't be concerned if the loss appears to plateau in the middle of pretraining; it will eventually go back down.)

```
# Pretrain the model
python src/run.py pretrain vanilla wiki.txt \
        --writing_params_path vanilla.pretrain.params

# Finetune the model
python src/run.py finetune vanilla wiki.txt \
        --reading_params_path vanilla.pretrain.params \
        --writing_params_path vanilla.finetune.params \
        --finetune_corpus_path birth_places_train.tsv

# Evaluate on the dev set; write to disk
python src/run.py evaluate vanilla wiki.txt  \
        --reading_params_path vanilla.finetune.params \
        --eval_corpus_path birth_dev.tsv \
        --outputs_path vanilla.pretrain.dev.predictions

# Evaluate on the test set; write to disk
python src/run.py evaluate vanilla wiki.txt  \
        --reading_params_path vanilla.finetune.params \
        --eval_corpus_path birth_test_inputs.tsv \
        --outputs_path vanilla.pretrain.test.predictions
```

   Report the accuracy on the dev set (printed by the third command above). We expect the dev accuracy will be at least 10%, and will expect a similar accuracy on the held out test set.

7. succinctly explain why the pretrained (vanilla) model was able to achieve an accuracy of above 10%, whereas the non-pretrained model was not

Acknowledgements This question was adapted from Stanford's CS224n course. Their contributions are greatly appreciated.